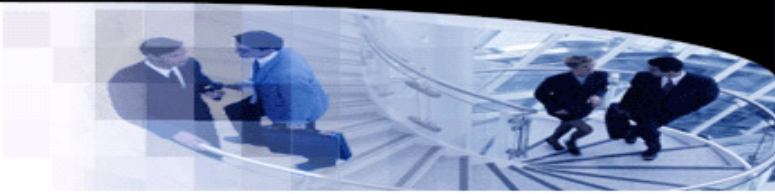
武漢大學
WUHAN UNIVERSITY

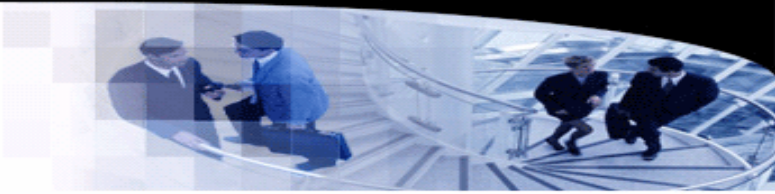


Programming Languages and Compilers

Lecture 28. Garbage Collection

袁梦霆 (YUAN Mengting)
ymt@whu.edu.cn
School of Computer Science, Wuhan University

Heap Management



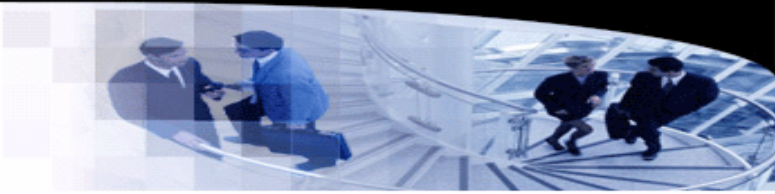
- User-managed heap
  - User-managed heaps are error prone.

```
class List {
  List *next;
  List() {
    next = nullptr;
  }
};
```

```
void foo() {
  List *p = bar();
  …
  delete p; // Double free!
}
```

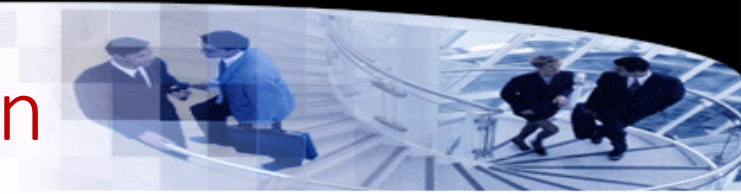
```
List *bar() {
  List *h = new List();
  h->next = new List(); // Memory leak!
  p = h;
  …
  delete p;
  …
  return h;
}
```

Garbage Collection



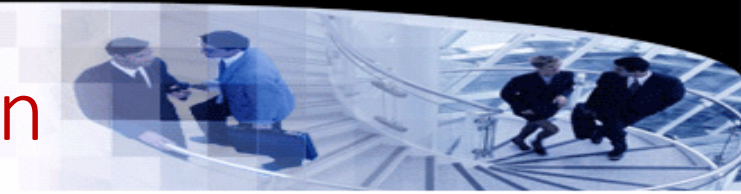
- Automatic memory management
  - A memory is called garbage, if it will no longer be used.
  - The garbage collector attempts to reclaim allocated garbage memory.
  - First introduced in Lisp.
  - Require no user deallocation.
- Pros
  - Less memory errors.
- Cons
  - Run-time overhead.

Reference Counting Garbage Collection



- Idea
  - For each object, use a counter to the references to this object.
  - The counter is set to 1 when an object is created.
  - Each time an object is passed as a parameter, increase the counter of the object.
  - Given an assignment u = v, increase the counter of v and decrease the counter of the object that was originally referenced by u.
  - Each time a function returns, decrease the counters of objects that are referenced by local variables.
  - Reclaim an object if the counter is 0.
  - When an object is reclaimed, decrease the counters of all its member objects.

Reference Counting Garbage Collection



- A Java style example

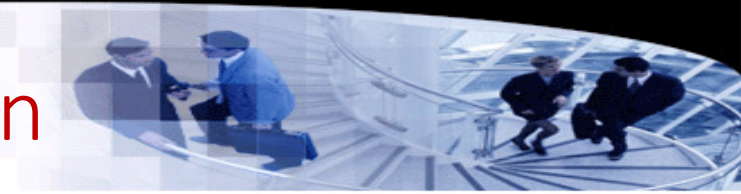
```
class List {
  List next;
};

int main() {
  List h = new List();




}
```

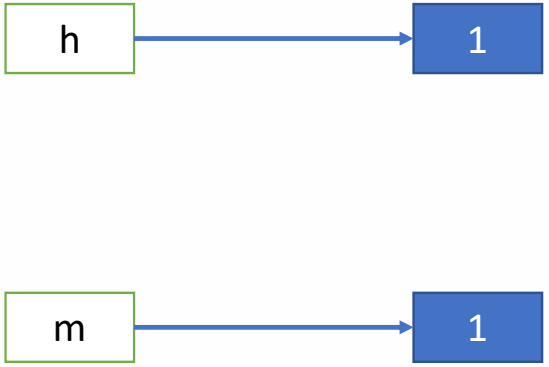
Reference Counting Garbage Collection



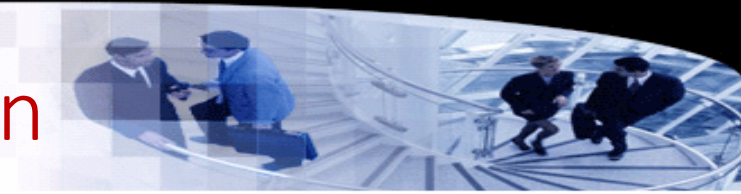
- A Java style example

```
class List {
  List next;
};

int main() {
  List h = new List();
  List m = new List();
```



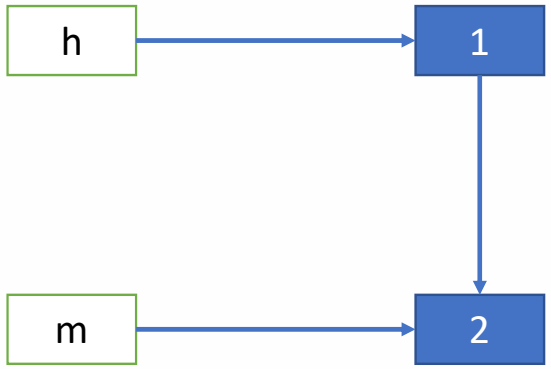
Reference Counting Garbage Collection



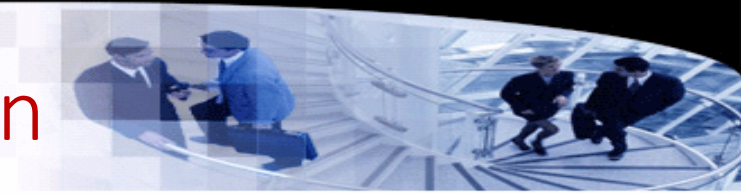
- A Java style example

```
class List {
  List next;
};

int main() {
  List h = new List();
  List m = new List();
  h.next = m;
```



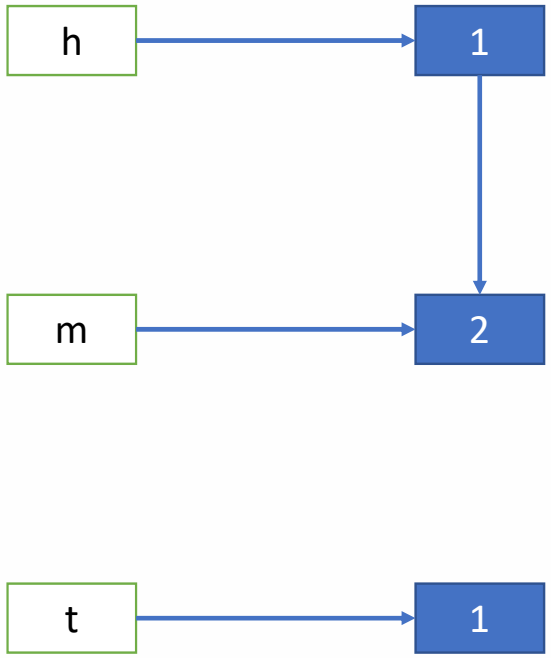
Reference Counting Garbage Collection



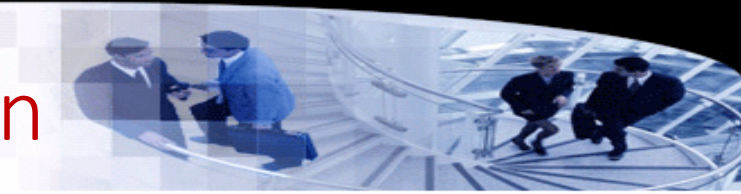
- A Java style example

```
class List {
  List next;
};

int main() {
  List h = new List();
  List m = new List();
  h.next = m;
  List t = new List();
```



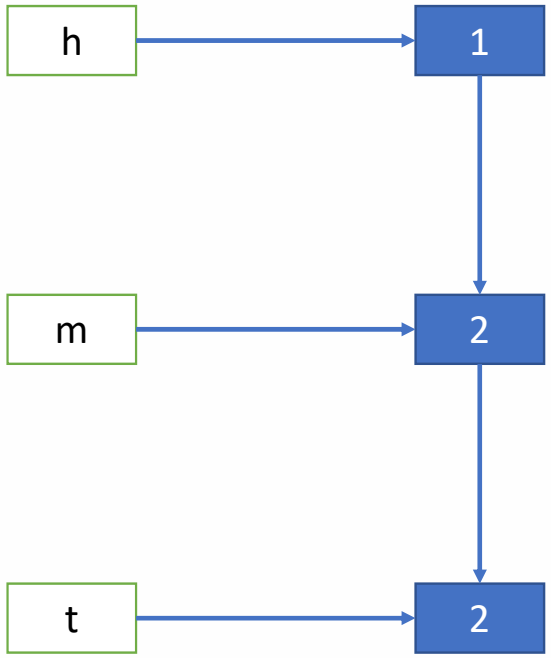
Reference Counting Garbage Collection



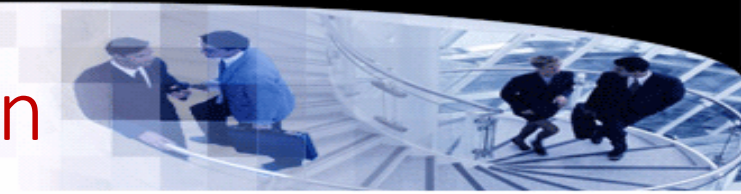
- A Java style example

```
class List {
  List next;
};

int main() {
  List h = new List();
  List m = new List();
  h.next = m;
  List t = new List();
  m.next = t;
```



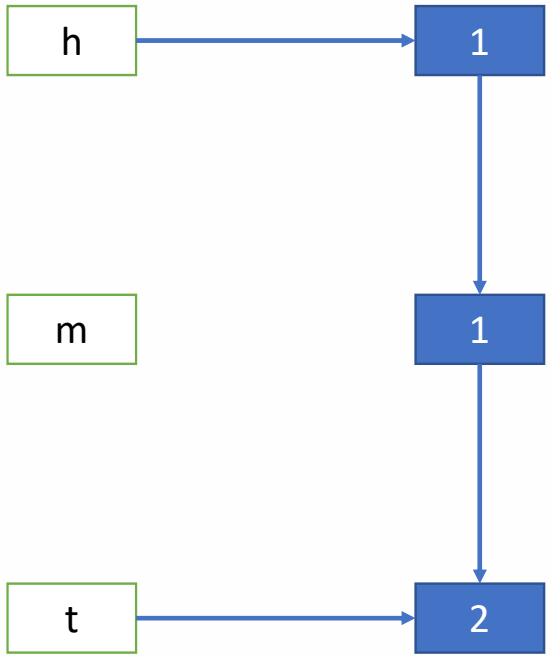
Reference Counting Garbage Collection



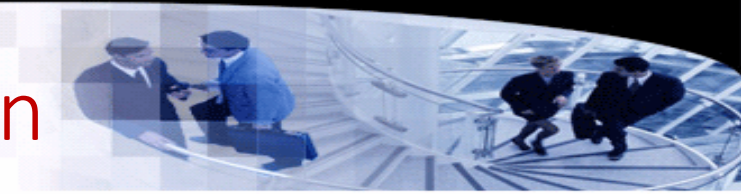
- A Java style example

```
class List {
  List next;
};

int main() {
  List h = new List();
  List m = new List();
  h.next = m;
  List t = new List();
  m.next = t;
  m = null;
```



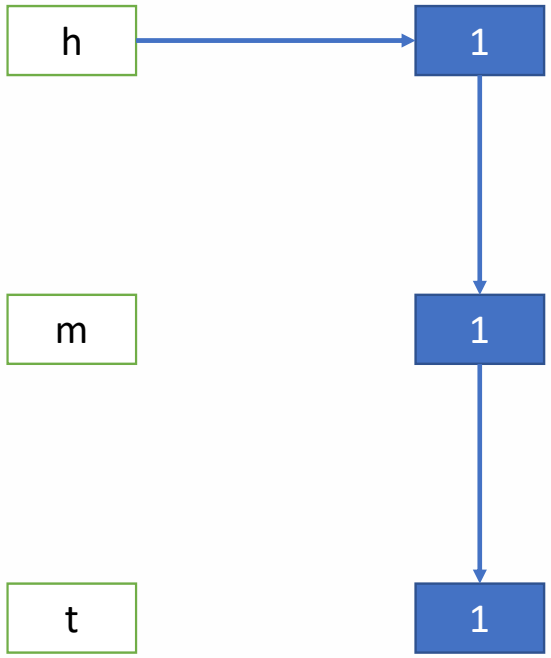
Reference Counting Garbage Collection



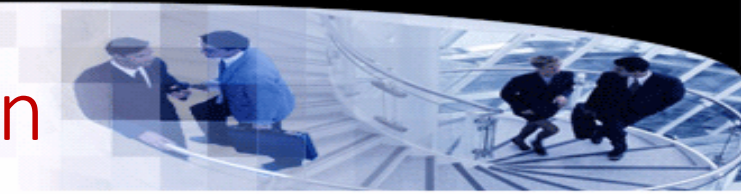
- A Java style example

```
class List {
  List next;
};

int main() {
  List h = new List();
  List m = new List();
  h.next = m;
  List t = new List();
  m.next = t;
  m = null;
  t = null;
```



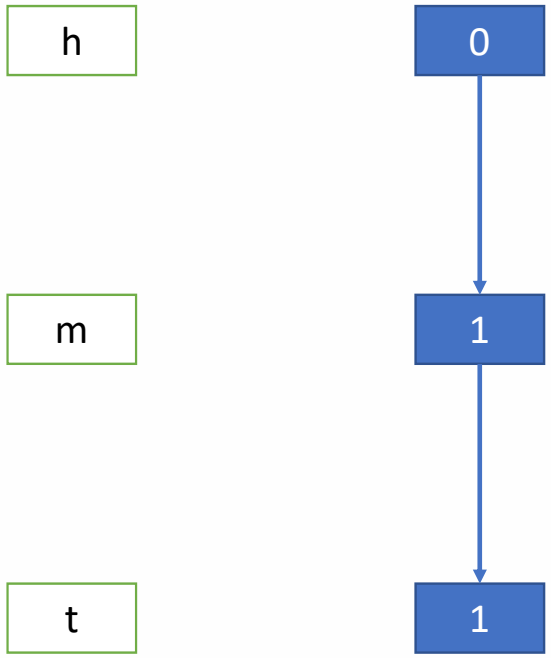
Reference Counting Garbage Collection



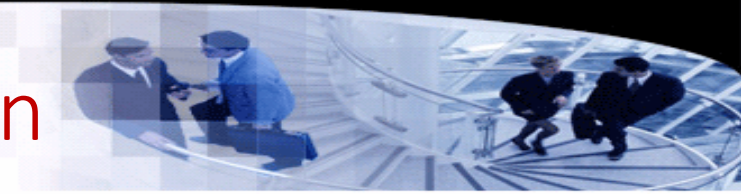
- A Java style example

```
class List {
  List next;
};

int main() {
  List h = new List();
  List m = new List();
  h.next = m;
  List t = new List();
  m.next = t;
  m = null;
  t = null;
}
```



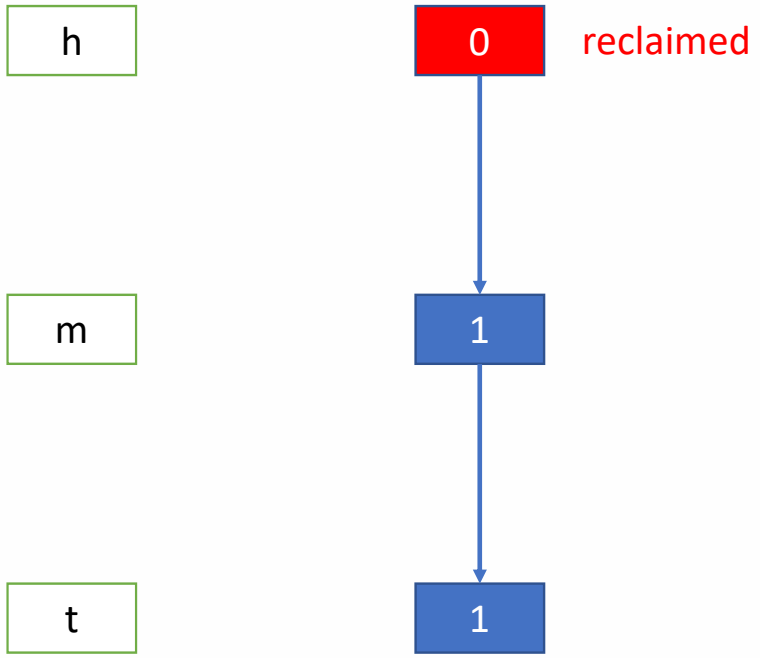
Reference Counting Garbage Collection



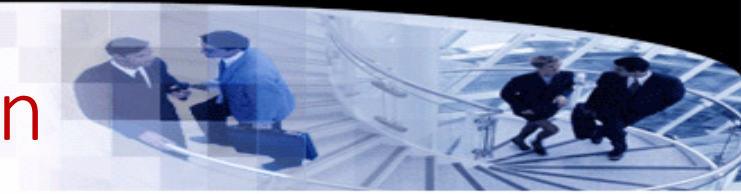
- A Java style example

```
class List {
  List next;
};

int main() {
  List h = new List();
  List m = new List();
  h.next = m;
  List t = new List();
  m.next = t;
  m = null;
  t = null;
}
```



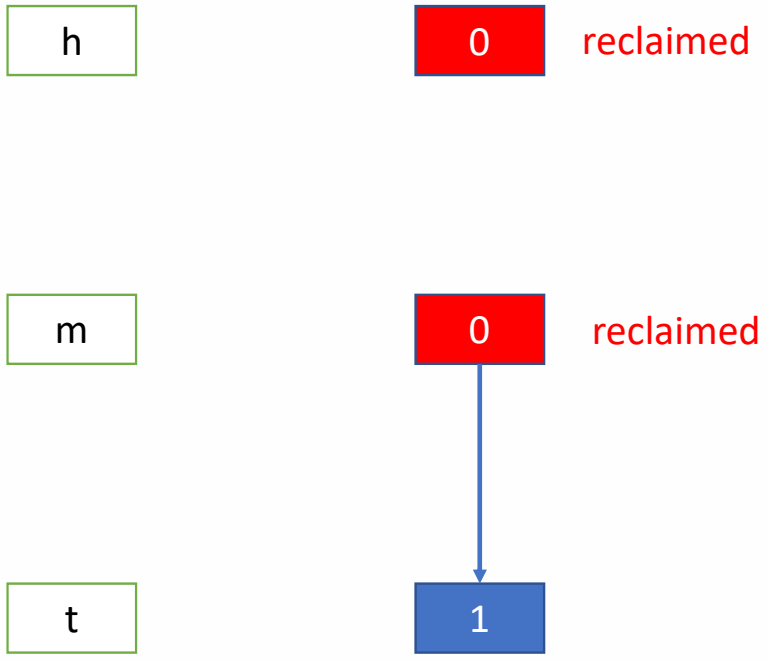
Reference Counting Garbage Collection



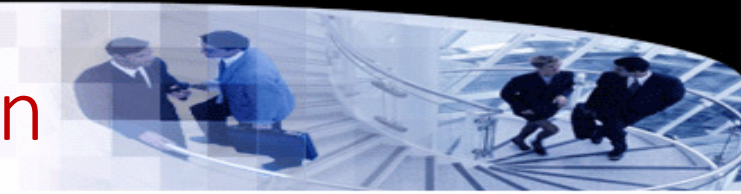
- A Java style example

```
class List {
  List next;
};

int main() {
  List h = new List();
  List m = new List();
  h.next = m;
  List t = new List();
  m.next = t;
  m = null;
  t = null;
}
```



Reference Counting Garbage Collection



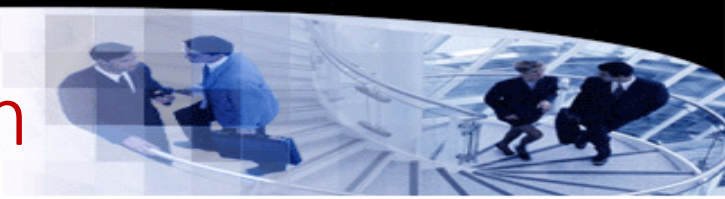
- A Java style example

```
class List {
  List next;
};

int main() {
  List h = new List();
  List m = new List();
  h.next = m;
  List t = new List();
  m.next = t;
  m = null;
  t = null;
}
```

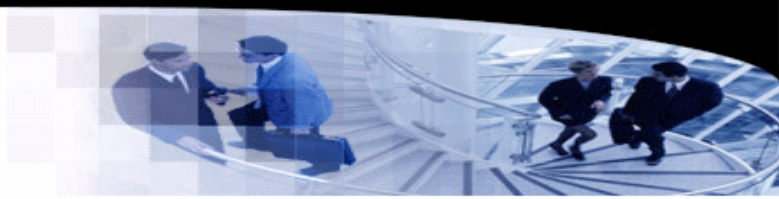
h	0	reclaimed
m	0	reclaimed
t	0	reclaimed

Reference Counting Garbage Collection



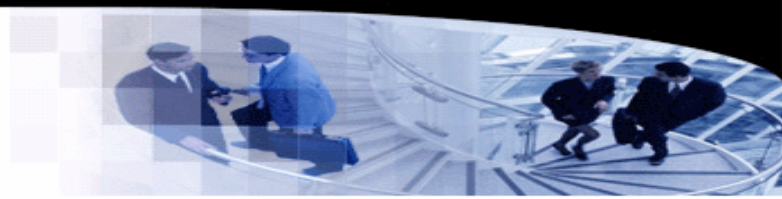
- Advantages
  - Incremental operations on counters.
  - Easy to implement.
- Disadvantages
  - Hard to handle cyclic references.
  - Extra counter storage for each object.

Mark-and-Sweep



- Basic idea
  - A periodic approach to garbage collection.
  - Pause the program.
  - Marking:
    - Find reachable objects from a **root set**.
    - Mark all reachable objects.
  - Sweeping:
    - Reclaim all unmarked objects.
- Obtaining the root set
  - Global (static) variables.
  - Local variables and variables from current stack frames.

Mark-and-Sweep

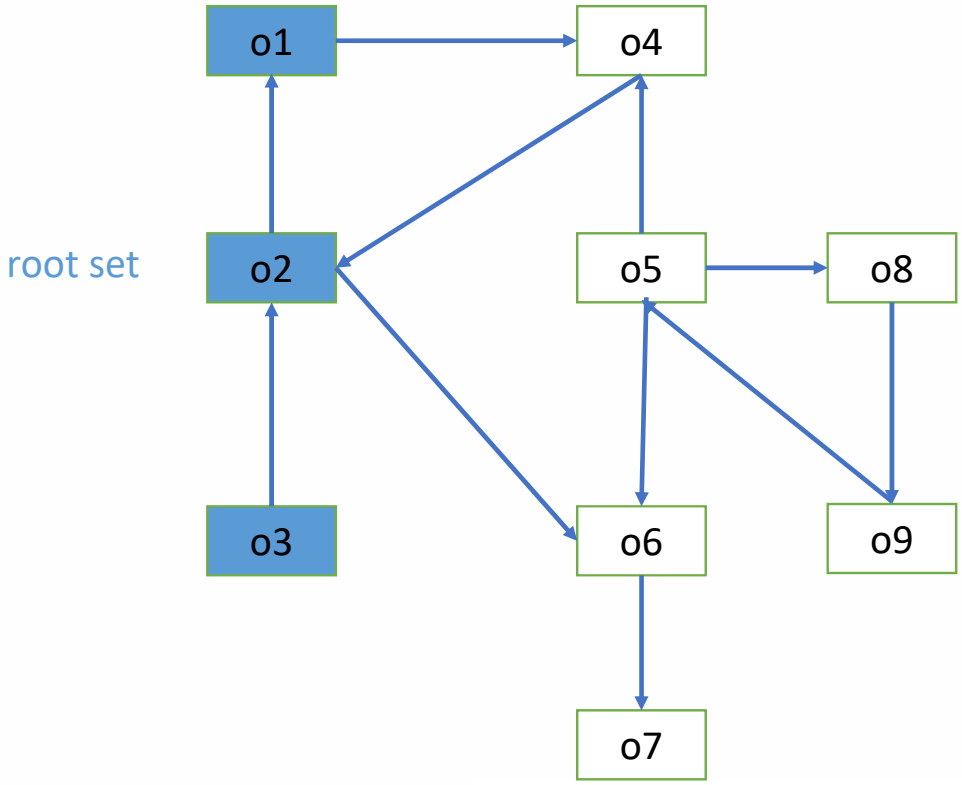


- Example

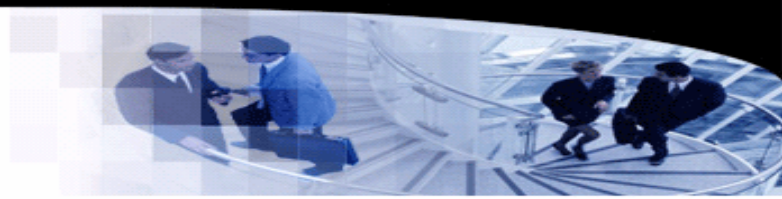
```
List a = List (); // o1
```

```
int main() {
  Object b = new List(); // o2
  …
  foo();
}
```

```
void foo() {
  Object c = new List(); // o3
  …
  // Mark-and-sweep now.
}
```



Mark-and-Sweep

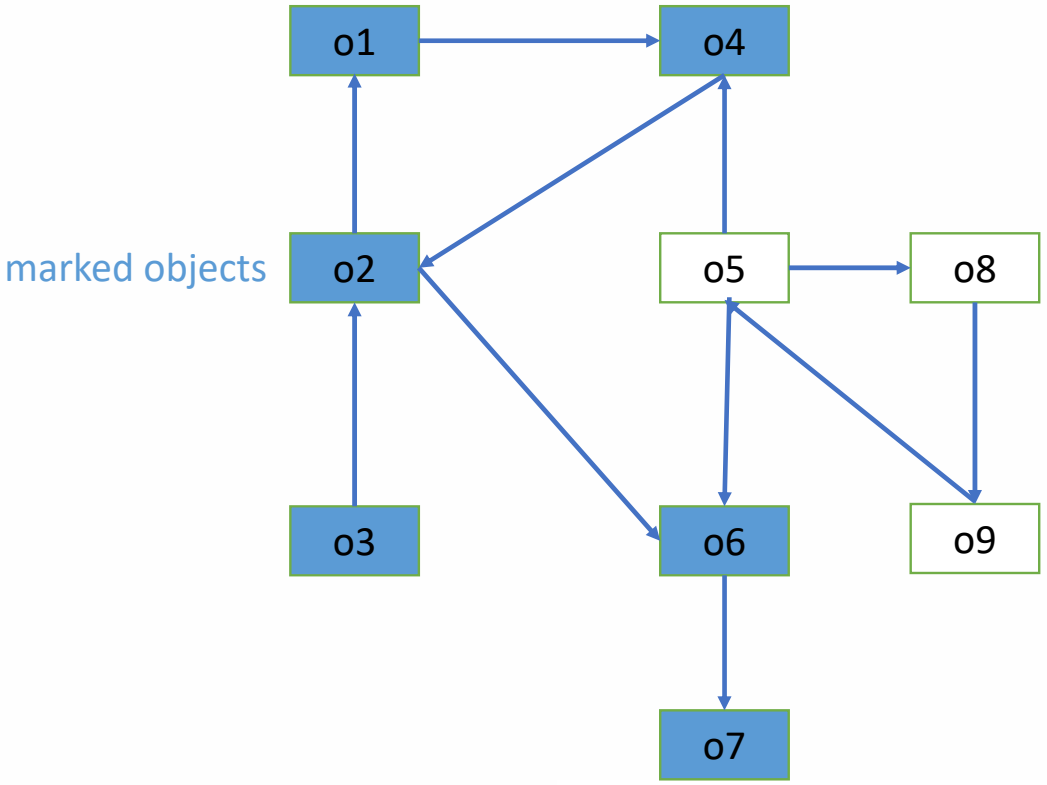


- Example

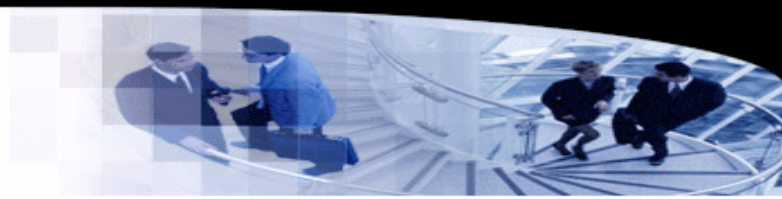
```
List a = List (); // o1
```

```
int main() {
  Object b = new List(); // o2
  …
  foo();
}
```

```
void foo() {
  Object c = new List(); // o3
  …
  // Mark-and-sweep now.
}
```



Mark-and-Sweep

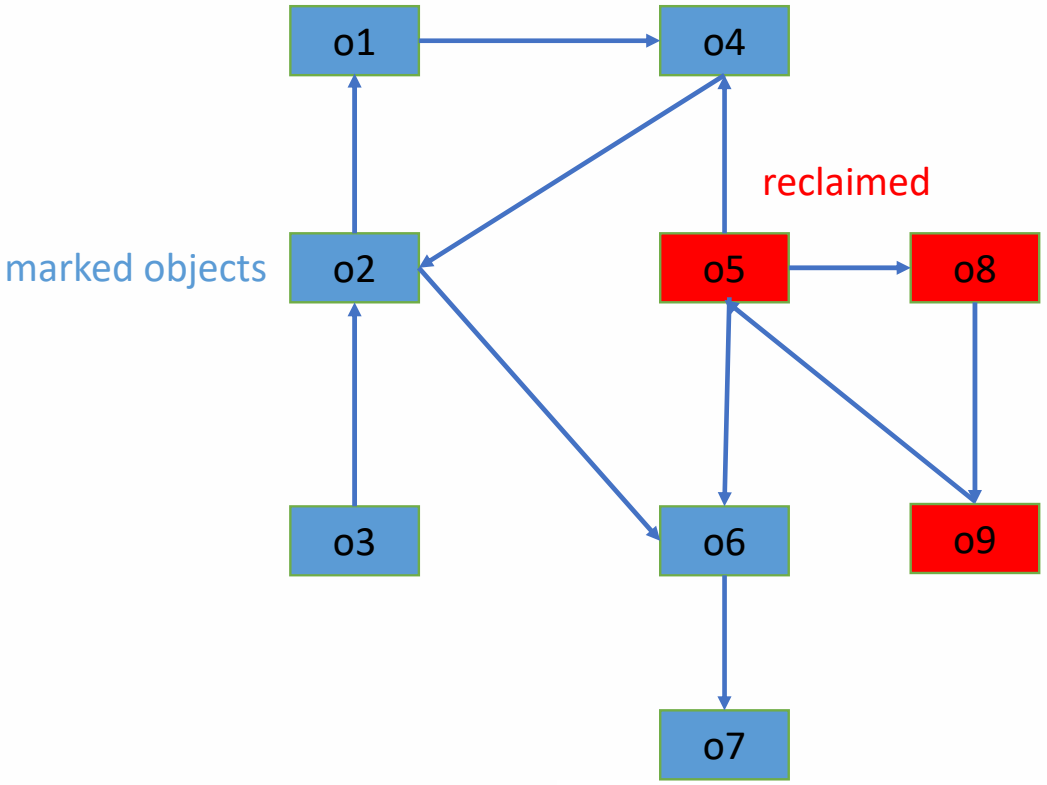


- Example

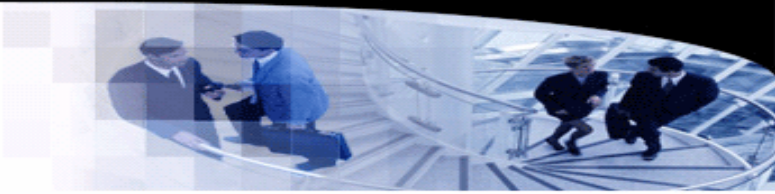
```
List a = List (); // o1
```

```
int main() {
  Object b = new List(); // o2
  …
  foo();
}
```

```
void foo() {
  Object c = new List(); // o3
  …
  // Mark-and-sweep now.
}
```

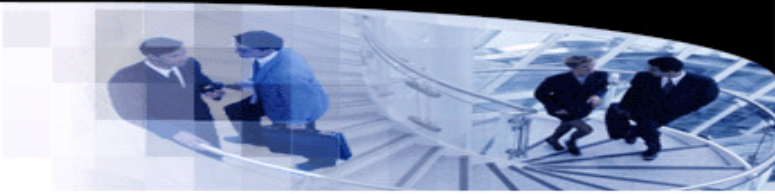


Mark-and-Sweep



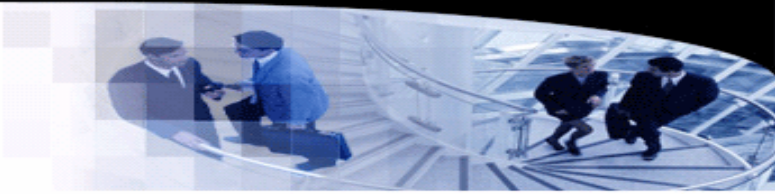
- Advantages
  - Capable of reclaim cyclic references.
  - Low overhead (1 bit per object).
  - The time complexity is proportional to the number of reachable objects.
- Disadvantages
  - The program should be paused to perform garbage collection.
  - Memory fragmentation.
  - Whole heap traverse.

Generational Garbage Collection



- Insight
  - Most objects have a very short life span.
  - The longer an object has lasted, the more likely it is to last to the end.
- Idea
  - Divide the heap memory into generations.
  - Once a generation if full, the allocator moves to a new generation.
  - Garbage collection is mostly performed on new generations.
  - Less frequent scans are performed for older generations.
  - A thorough scan is occasionally performed on all generations to reclaim more garbage objects.

Copying Collection



- Idea
  - Separate the heap memory into two halves.
  - Perform mark-and-sweep on one half.
  - Copy marked object to the other half.
  - Compact data while copying, and adjust references.
  - The next garbage collection will happen on the other half.
- Advantages
  - Less fragmentations.
- Disadvantages
  - Less heap memory to allocate.